VALOR / ShrikeFi — Judge Study Notes

> **Purpose:** one plain-language file to read before the demo and to answer judge questions without getting lost in code.
>
> **Project:** SIH26181 — VALOR (Vital and Atmospheric Logic for Offline Rescue)
>
> **Hardware:** Vicharak ShrikeFi — ESP32-S3 + Renesas ForgeFPGA SLG47910C
>
> **Last updated:** 2026-10-19. Current ForgeFPGA bitstream is the 25 % crest/peak-4 build (`d9d58daa…` , 46,408 bytes), flashed on COM5.

Quick jump: 30-second pitch · System diagram · Sensors · I2C/SPI/UART · Code layers · Data flow · FPGA · Algorithms · Quality gates · Fall/SOS FSM · Outputs · Verified vs not · Judge Q&A · Demo script · Cheat sheet

---

1. The 30-second explanation

> “VALOR is a wearable disaster-triage companion. It measures the person and the environment at the same time: heart rate, SpO_2, HRV and respiratory rate from a PPG sensor; temperature, humidity and pressure from a BME280; and PM2.5 from a laser particulate sensor. A small FPGA finds each heartbeat in hardware with 20 ns timing resolution, and an ESP32-S3 runs the clinical and disaster-risk logic. It works completely offline, shows an action on the OLED, and also serves a local web dashboard.”

If the judge only remembers one sentence:

> **“Health plus environment, fused on the edge, offline — not a cloud wearable.”**

---

2. The problem we chose

A normal fitness band answers “how hard did I exercise?”
A disaster worker needs four different answers at once:

1. Is the heart under strain?
2. Is blood oxygen falling?
3. Is the air toxic?
4. Is the environment about to become more dangerous than the body can handle?

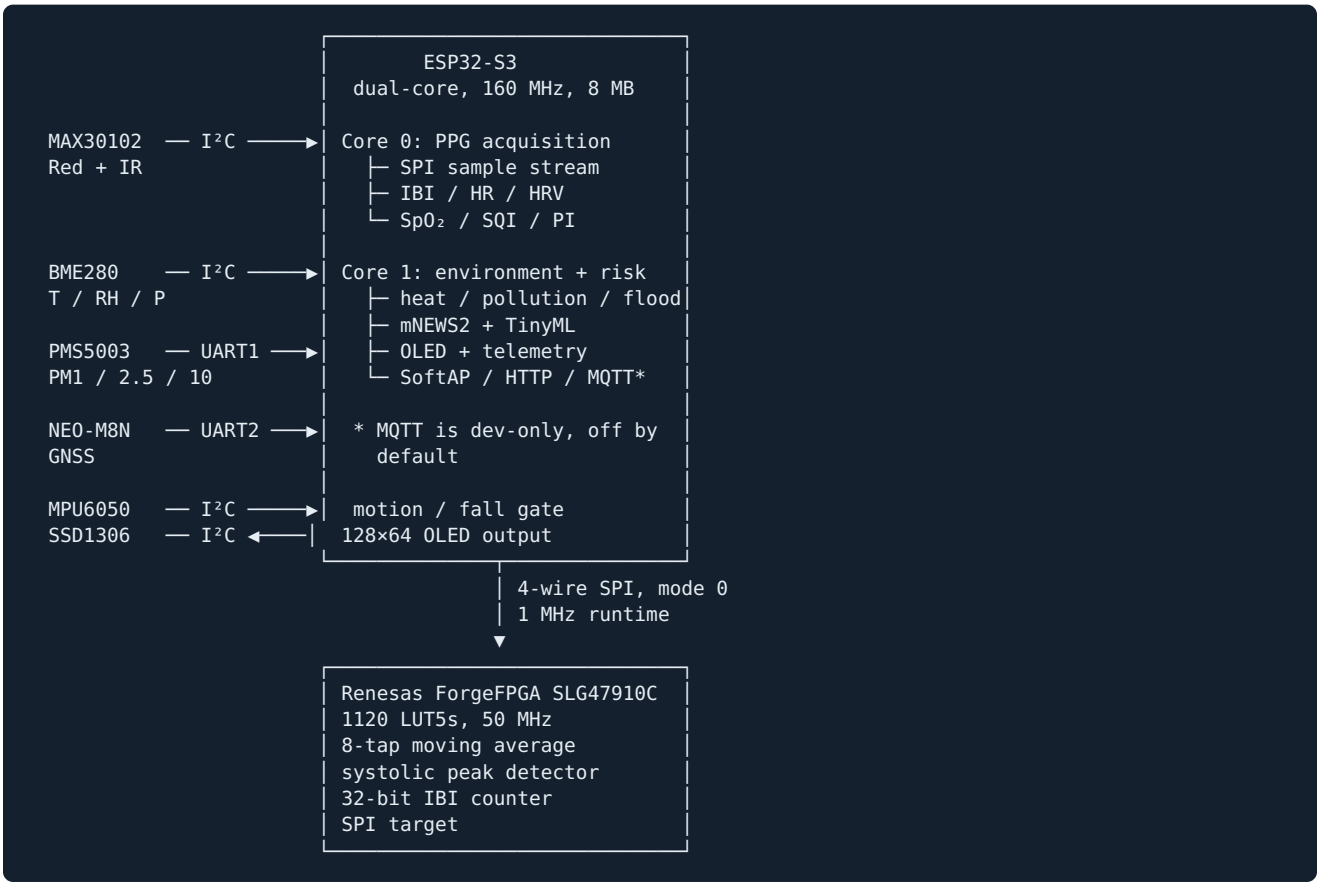
No single sensor answers all four. The project’s core idea is **fusion**: PPG + environment + motion + location, with a triage decision made **on the device**, not in the cloud.

Typical users in the pitch:

- disaster first responders
- flood / cyclone evacuees
- miners and industrial workers
- rural or low-resource clinics

---

3. System at a glance



Two-core split

Core	Main job	Why
Core 0	100 Hz PPG acquisition, SPI link, IBI, HRV, SpO_2	Keeps the timing-critical loop away from Wi-Fi/HTTP
Core 1	BME280, PMS5003, GNSS, IMU, risk engines, TinyML, OLED, web	Radio and UI can jitter without disturbing the PPG loop

Shared data is protected by a FreeRTOS mutex (`s_data_mutex`).

---

4. Hardware

Main parts

Part	What it is	Role
ESP32-S3-WROOM-1-N8R2	Dual-core Xtensa LX7, 8 MB flash, Wi-Fi/BLE	Main processor, networking, UI
Renesas ForgeFPGA SLG47910C	Small FPGA, 1120 5-input LUTs	PPG filtering, peak detection, IBI timing
MAX30102	Red + IR optical sensor	PPG → HR, SpO_2, HRV, perfusion
BME280	Temperature, humidity, pressure	Heat index, pressure trend, PM correction
PMS5003 / PMSA003	Laser particulate sensor	PM1.0 / PM2.5 / PM10

Part	What it is	Role
NEO-M8N	GNSS receiver	Location / rescue context
MPU6050	6-axis IMU	Motion artifact gate, fall detection
SSD1306	128×64 OLED	Offline advisory display
Active buzzer	GPIO 4	Audible alert
SOS cancel button	GPIO 5	5 s hold to cancel an alert; firmware ready, physical button still pending

Power / voltage — very important

- **Every GPIO and every logic signal is 3.3 V.**
- **5 V is only for the PMS5003 VCC rail** (fan + laser), from USB-C VBUS.
- **Never put 5 V on an ESP32-S3 or ForgeFPGA GPIO.** It permanently damages the parts.
- Common ground is required.
- The prototype is **USB-powered**; no battery/charger is fitted. We have **not** measured battery runtime or mW; do not quote a battery figure.

---

5. The three buses - I2C, SPI, UART

This is one of the most likely judge questions. Keep the answer simple:

> "Use the smallest bus that fits the job: I²C for short-board sensors, SPI for the deterministic FPGA link, UART for sensors that stream their own data."

5.1 I²C — sensors on the board

Item	Value
Pins	**GPIO 1 = SDA**, **GPIO 2 = SCL**
Speed	**100 kHz**
Logic	3.3 V
Pull-ups	Typically on the breakout boards; effective bus pull-up should be roughly 1.5–3.3 kΩ

Address	Device	Data
`0x3C`	SSD1306 OLED	output only
`0x57`	MAX30102	Red + IR PPG
`0x68`	MPU6050	acceleration / gyro
`0x76`	BME280	temperature / humidity / pressure
`0x48`	optional MAX30205	not fitted on this build

Why 100 kHz instead of 400 kHz? Multi-drop breadboard wiring and several breakouts can produce NACKs at higher speed. The sensor bandwidth does not need 400 kHz.

5.2 SPI — ESP32 to ForgeFPGA

Signal	ESP32 GPIO	FPGA side
`SCK`	**GPIO 12**	`PIN_16`

Signal	ESP32 GPIO	FPGA side
`SS_n` / CS	**GPIO 10**	`PIN_17`
`MOSI`	**GPIO 11**	`PIN_18`
`MISO`	**GPIO 13**	`PIN_19`
FPGA power	**GPIO 9**	`PWR`
FPGA enable	**GPIO 8**	`EN`

- 4-wire SPI, **mode 0** (CPOL = 0, CPHA = 0).
- **1 MHz** during normal runtime.
- **16 MHz** only while streaming the bitstream at boot.
- **One 8-bit transaction per sample.**
- No interrupt pin. The beat is **bit 7 of the byte coming back on MISO**.
- No command map and no address register. The link is deliberately simple.

```
MCU → FPGA : [ raw 8-bit IR sample ]
MCU ← FPGA : [ beat | filtered[6:0] ]
                bit 7   bits 6:0
```

When a beat is detected, the FPGA queues a **5-byte IBI burst**:

```
0xC3 | IBI[31:24] | IBI[23:16] | IBI[15:8] | IBI[7:0]
```

The IBI is in **50 MHz cycles**, so one tick is **20 ns**. The ESP32 reads the burst; if the magic byte is absent (older bitstream), it falls back to its own microsecond timer.

Important honest detail: the returned waveform field is only 7 bits, so it wraps at 128 and looks like a sawtooth in logs. The FPGA's internal detector uses the full 8-bit value, so detection is not affected. Do not call the returned 7-bit byte a clean waveform.

5.3 UART — streaming sensors

UART	Pins	Baud	Device
UART1	RX **GPIO 14**, TX GPIO 18 (not wired)	**9600**	PMS5003 particulate sensor
UART2	RX **GPIO 16**, TX **GPIO 17**	**9600**	NEO-M8N GNSS, NMEA-0183
USB Serial/JTAG	USB-C	**115200**	PC log / dashboard, COM5

- PMS5003 VCC is **5 V**; its TX/RX logic is 3.3 V, so it connects directly to the ESP32.
- The firmware only receives from the PMS; the TX line is not wired.
- The GNSS streams NMEA sentences; the firmware parses location.
- The USB serial link is how the PC dashboard sees live telemetry.

5.4 GPIOs used

GPIO	Use
4	active buzzer
5	SOS cancel button (5 s hold)
8	FPGA enable
9	FPGA power
10	FPGA SPI CS
11	FPGA SPI MOSI

GPIO	Use
12	FPGA SPI SCK
13	FPGA SPI MISO
14	PMS5003 UART RX
16	GNSS UART RX
17	GNSS UART TX
1 / 2	I²C SDA / SCL

6. Code structure — the layers

Think of the repo in six layers:

Layer 1 — Hardware / electrical

PCB traces, 3.3 V rules, pull-ups, sensor breakout boards, FPGA power/enable.

Layer 2 — Sensor drivers

`firmware/shrikefi/max30102.c`, `bme280.c`, `pms5003.c`, `gnss`/NMEA parser, `mpu6050.c`, `ssd1306.c`, `esp32_i2c_hal.c`.

These talk the sensor's own protocol (I²C or UART), handle recovery, and expose clean values.

Layer 3 — Transport / acquisition

`firmware/shrikefi/shrikefi_link_driver.c` — SPI2 link to the FPGA; bitstream flashing; beat flag; IBI burst.

`firmware/shrikefi/main_shrikefi.c` — Core 0 acquisition task; Core 1 monitor task; shared state.

Layer 4 — Signal processing / FPGA

`hardware/common/moving_average_8tap.v` and `hardware/common/ppg_peak_detector.v` — vendor-agnostic RTL shared by the Zynq baseline and the ForgeFPGA port.

`hardware/shrikefi/forgefpga_ppg_top.v`, `spi_target.v` — ForgeFPGA top and SPI target.

Layer 5 — Clinical / risk logic

`firmware/core/`:

- `hrv_analysis.c` — RMSSD / SDNN / median reference
- `spo2_engine.c` — ratio-of-ratios SpO_2
- `ppg_sqi.c` — signal quality
- `ppg_respiratory_rate.c` — RSA respiratory rate
- `clinical_vitals_engine.c` — mNEWS2-style triage
- `disaster_risk_engine.c` — heat / pollution / flood / cyclone
- `pressure_trend.c` — barometric trend
- `nn_risk_model_int8.c` — 619-byte TinyML model

Layer 6 — Presentation / connectivity

- `firmware/shrikefi/web_status.c` — SoftAP status page + JSON + 24-slot history
- `firmware/shrikefi/dashboard/valor_dashboard.html` — standalone live dashboard, no CDN
- `wifi_mqtt_manager.c` — optional dev MQTT (off by default)

- OLED rendering inside `main_shrikefi.c`

Where to look if a judge points at the code

If they ask…	Open…
“Where is the FPGA logic?”	`hardware/common/ppg_peak_detector.v`, `moving_average_8tap.v`, `hardware/shrikefi/forgefpga_ppg_top.v`
“Where is the SPI protocol?”	`firmware/shrikefi/shrikefi_link_driver.c`, `spi_target.v`
“Where are the tasks?”	`firmware/shrikefi/main_shrikefi.c` (`task_ppg_accelerator`, `task_disaster_monitor`)
“Where is HRV?”	`firmware/core/hrv_analysis.c/.h`
“Where is SpO₂?”	`firmware/core/spo2_engine.c/.h`
“Where is the risk logic?”	`firmware/core/clinical_vitals_engine.c`, `disaster_risk_engine.c`
“Where is the fall FSM?”	`firmware/shrikefi/fall_detect.c/.h`, `test_fall_detect.c`
“Where is the SOS FSM?”	`firmware/shrikefi/sos.c/.h`
“Where is the AI?”	`firmware/core/nn_risk_model_int8.c`, `train_nn_risk_model.py`
“Where is the web page?”	`firmware/shrikefi/dashboard/valor_dashboard.html`
“Where is the history?”	`firmware/shrikefi/web_status.c`
“Where are the tests?”	`hardware/*/tb_*.v`, `firmware/zynq/test_*`, `run_host_test.bat`

7. Boot sequence — what happens when we power it on

1. **ESP32-S3 boots** under ESP-IDF/FreeRTOS.
2. **I²C bus starts** at 100 kHz; firmware probes the OLED, MAX30102, BME280 and MPU6050.
3. **OLED initializes**, then the firmware brings up PMS5003 UART, GNSS UART and the IMU task.
4. **FPGA is programmed by the ESP32**, not by a separate programmer: - drive `PWR` / `EN` / `SS` through the vendor boot-mode sequence; - stream the **46,408-byte** bitstream over SPI2 at **16 MHz** in 256-byte chunks; - release the bus; switch to **1 MHz** runtime SPI.
5. **Core 0 starts PPG acquisition** and the SPI link.
6. **Core 1 starts risk processing**, environment, OLED, telemetry and web.
7. Once a finger is present, the FPGA starts returning beats and IBIs; HR appears after the priming intervals (typically 6–8 s), SpO₂ later (roughly 15–25 s).

Honest FPGA answer: the bitstream transfer is open-loop. The boot log says `configuration COMPLETE! (46408 bytes loaded)`, but that only means the bytes were clocked out. The real proof that the FPGA is running is a `[FPGA ACCEL] Systolic crest detected!` line after real sensor contact. The `0x55` probe returning `0x00` is expected, not a handshake.

8. Data flow — from light to advisory

```
1. MAX30102 FIFO (100 Hz)
   └ Red + IR samples over I²C
        |
2. Core 0
   ├ pre-smooths / converts IR to 8-bit
   ├ sends one 8-bit sample per SPI transaction
   └ receives beat flag + filtered value
        |
3. ForgeFPGA
   ├ 8-tap moving average (O(1) running sum)
   ├ systolic peak FSM + 25 % crest confirmation
   ├ 32-bit IBI counter at 50 MHz
   └ 5-byte IBI burst after each beat
        |
4. Core 0 pipeline
   ├ rejects implausible intervals (range + median/pair rules)
   ├ HR from accepted IBIs
   ├ HRV: 300-beat RMSSD window
   ├ SpO₂ from Red/IR ratio
   └ SQI / PI from the raw optical window + IMU motion
        |
5. Shared state under `s_data_mutex`
        |
6. Core 1
   ├ BME280: heat index, pressure trend
   ├ PMS5003: PM2.5 → pollution risk
   ├ MPU6050: motion / fall
   ├ mNEWS2 clinical score
   ├ environmental rule engine + TinyML
   ├ fuse: worst risk wins, critical vital overrides
   └ OLED / USB telemetry / SoftAP / optional MQTT
```

What each output means

Output	Meaning
HR	Heart rate from accepted beat intervals
SpO₂	Estimated oxygen saturation from Red/IR ratio
PI	Perfusion index: pulsatile AC / steady DC, in %
SQI	Confidence in the PPG window, 0–1
RMSSD	Beat-to-beat variability; higher generally means more parasympathetic activity
RR	Respiratory rate estimated from RSA in the IBI series
mNEWS2	Published NHS-style early-warning score, not a diagnosis
Risk	Heat / pollution / flood / cyclone level + advisory

9. The FPGA — what it does and why

Why use an FPGA at all?

The ESP32-S3 could do a moving average and peak detector in software. The reason for the FPGA is **determinism**:

- 50 MHz hardware clock → **20 ns** timing resolution
- no FreeRTOS scheduling jitter
- no Wi-Fi/HTTP interrupt in the beat-timing path

- the ESP32 is free for clinical logic, display and radio

The same RTL also runs on the Zynq-7000 baseline (AXI4-Lite version), so the algorithm is not tied to one vendor.

8-tap moving average

Instead of adding 8 samples every time:

```
new_sum = old_sum + new_sample - oldest_sample
output  = new_sum >> 3
```

That is two additions and a wire shift, regardless of filter length.

- 8-bit samples → 11-bit accumulator → no overflow
- `>> 3` is an exact divide by 8
- **0 DSP slices, 0 BRAMs**

Peak detector

Four-state FSM:

State	What it does
`ARMED`	waits for the signal to cross the threshold and rise
`RISING`	tracks the running maximum
`PEAK_FOUND`	confirms a crest, latches the IBI, pulses the beat flag
`REFRACTORY`	blanking window to reject the dicrotic notch / double count

- Refractory: **250 ms** (safe for a resting/normal heart; a 240 bpm ceiling).
- Crest confirmation: current build requires a **25 % fall from the running peak** (`peak/4`). This was raised from 12.5 % because live captures still showed short/long split pairs.
- The FPGA also queues the 32-bit IBI burst.

Current ForgeFPGA fit (2026-10-19)

Resource	Value
LUT5s	**515 / 1120 — 45.98 %**
Flip-flops	**303**
CLBs	**105 / 140 — 75.00 %**
Free CLBs	**35**
BRAMs / PLL	0 / 8, 0 / 1
Achievable period	**11,356 ps**
Fitter-reported frequency	**88.059 MHz**
Margin at 50 MHz	**+8.644 ns**

The WNS in the report is **−9.357 ns**, but that is measured against an auto-generated 500 MHz constraint because the design declares no real clock constraint. **Do not call it timing closure.** At the documented 50 MHz, the margin is +8.644 ns, which is a good sign, not a formal closure.

Zynq baseline (for comparison)

Metric	Value
Part	`xc7z020clg400-1`, Vivado ML 2022.2 OOC

Metric	Value
LUTs	**185**
LUTRAM	16
FFs	266
DSP / BRAM	**0 / 0**
WNS / hold	**+5.603 ns / +0.184 ns**
Fmax	**69.45 MHz** against 50 MHz
Tests	**6/6**

The Zynq design is the verified baseline; the ForgeFPGA is the low-cost wearable port. The same `hardware/common/` RTL is shared.

---

10. Algorithms — in plain language

Heart rate

- Beat intervals come from the FPGA.
- The firmware rejects obviously impossible intervals and uses a causal median as the local rhythm reference.
- HR is derived from accepted intervals; a short/long pair should not be allowed to masquerade as a real rate change.

SpO₂ — ratio of ratios

The MAX30102 gives Red and IR. For each 1-second window:

```
R = (AC_red / DC_red) / (AC_ir / DC_ir)
SpO₂ ≈ 101.8 − 6.67 × R     (rough two-point calibration on this optical path)
```

- 8-window moving average (about 8 s).
- The reading is published only after the estimate is stable, so it takes roughly 15–25 s.
- This is **not clinical-grade**; it was roughly calibrated against a fingertip oximeter and needs proper calibration with a printed reflective cradle.

Perfusion index (PI)

```
PI = (AC_ir / DC_ir) × 100 %
```

- Low PI = weak pulse / poor contact.
- PI is published only when SQI is good and the value is in a sane range.
- It is not a diagnosis; it is a contact/perfusion indicator.

SQI — can we trust this window?

Composite of:

- perfusion index
- waveform skewness
- interval regularity
- IMU motion energy

Threshold: **SQI ≥ 0.70** for clinical use.
Below that, the device holds/qualifies the reading instead of inventing a value. A critical vital can still override the hold.

HRV — RMSSD

```
RMSSD = sqrt( mean( (IBI[k+1] − IBI[k])² ) )
```

- 300-beat rolling window (about 4 minutes at 75 bpm).
- Minimum 10 accepted intervals before publishing; readiness also requires contiguous valid pairs.
- **Current limitation:** split pairs from the dicrotic notch and startup/contact transients can still inflate RMSSD. The current peak-4 bitstream removed the old 888/645-style signature in the last capture, but the first 95 s still had a 2.38 s startup gap and clean-tail RMSSD ~46 ms. **Do not present RMSSD as clinically validated.** ECG/chest-strap comparison is still pending.
- This is also why the number can look “unstable” for the first few minutes: the 300-beat window is still filling.

Respiratory rate (RR)

- Estimated from **RSA** (respiratory sinus arrhythmia) in the IBI series, following the Charlton 2018 approach.
- Uses a rolling window of accepted beats and reports confidence/depth.
- Finger PPG RSA is weak; RR is often **“not resolved”**. Do not quote a respiratory rate as validated unless the confidence gate is open.
- A dedicated respiration sensor is the honest next step if RR is a headline requirement.

mNEWS2 — clinical triage

Published NHS National Early Warning Score idea:

- points for HR, SpO_2, respiratory rate, temperature, consciousness
- aggregate score → normal / elevated / high / critical
- specific alerts for hypoxia, tachycardia, bradycardia, tachypnea, bradypnea, autonomic shock, signal noise, hypothermia

We use it as a **triage/advisory** score, not a diagnosis. Body temperature is currently **not instrumented**, so the temperature term is not scored.

Environmental risk engines

Hazard	Inputs	What it means
Heat	NOAA heat index, humidity, HR, RMSSD, Moran PSI	physiological heat strain, not just air temperature
Pollution	PM2.5, SpO_2, HR, RMSSD, AHA autonomic strain	smog / particulate exposure
Flood / cold	pressure trend, ambient cold-stress proxy, HR, RMSSD	exposure risk; no skin-temperature sensor, so capped at HIGH
Cyclone / storm	barometric pressure **rate of fall**	falling pressure precedes a storm; a rise is not scored

TinyML

- Architecture: **6 → 24 → 16 → 3**
- Inputs: HR, RMSSD, SpO_2, temperature, humidity, PM2.5
- Outputs: heat score, pollution score, flood score

- **INT8**, **619 parameters / 619 bytes**
- It is **knowledge distillation**: trained to reproduce the rule engine's labels, not discovered from patient outcomes.
- The rule engine remains the explainable source of truth; the NN is a compact approximation. The final verdict takes the **more severe** of rule and NN, then fuses clinical + environmental risk.

Fusion rule

- The more severe of clinical and environmental risk wins.
- An unknown environmental assessment is not turned into "normal".
- A measured critical vital can override a low-SQI hold.
- The device reports "UNKNOWN / not assessed" rather than inventing normal values.

11. Quality gates and honesty rules

These rules matter because judges will ask whether the device can "fake" readings:

Gate	Rule	Why
PI (perfusion index)	Published only when SQI ≥ 0.70 **and** AC/DC value is 0.02–20 %.	Prevents publishing PI from noise or no-contact readings. Transient PI spikes are suppressed.
Contact inference	Contact is inferred from accepted HR + SQI, **not** from packet presence. Packet presence alone does not mean a finger is on the sensor.	Stops the dashboard from showing "live" just because serial bytes are arriving.
SpO_2	Published only after ~15–25 s of stable estimation. Published only when SQI is good.	Prevents publishing an unstable early calibration value.
RMSSD / HRV	Requires 10 accepted IBIs, contiguous valid pairs, SQI ≥ 0.70.	Prevents publishing HRV from artifact-filled or disconnected data.
RR	Reports "RSA not resolved" when unresolved. Never fakes a breathing rate.	Finger PPG RSA is weak; inventing a rate would be dangerous.
Environment	Shows `--` when missing or zero. Never shows `0.0 °C`, `0 %`, or `Good` for an absent sensor.	Prevents false "normal" readings when a sensor is unplugged.
Critical override	A measured critical vital (e.g. SpO_2 ≤ 85 %) can override a low-SQI hold.	A real emergency is never silenced by a quality gate.

Honest language to use:

- "We hold rather than fabricate a value."
- "If the sensor is absent or the signal is poor, we show `--` or `UNKNOWN`, not a reassuring number."
- "Critical vitals can still override the SQI gate — an actual emergency is not hidden."

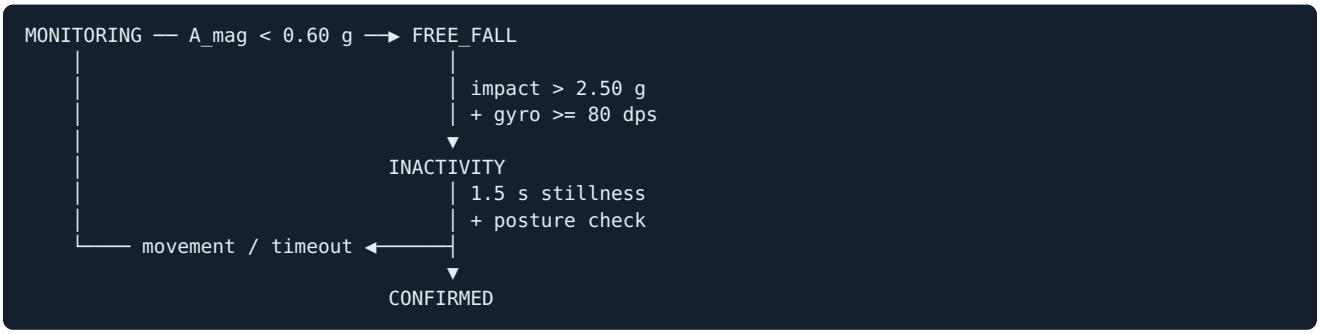
---

12. Fall detection and SOS FSM — yes, it is implemented

There are **two separate state machines**:

1. **Fall detector** — `fall_detect.c/.h`, a 5-state biomechanical FSM.
2. **Emergency/SOS state machine** — `sos.c/.h`, a 4-state escalation FSM.

Fall detector FSM



State	Meaning
`MONITORING`	normal activity; tracks the gravity/orientation baseline
`FREE_FALL`	weightlessness detected (`A_mag < 0.60 g`)
`IMPACT`	high deceleration shock (`A_mag > 2.50 g` , gyro ≥ 80 dps)
`INACTIVITY`	post-impact stillness/incapacitation check
`CONFIRMED`	fall latched; SOS is armed

Key timings and thresholds:

Parameter	Value
Free-fall threshold	**< 0.60 g**
Impact threshold	**> 2.50 g**
Rotational impact gate	**≥ 80 dps**
Free-fall-to-impact window	**800 ms**
Post-impact rebound settling	**400 ms**
Stillness duration to confirm	**1.5 s**
Stillness tolerance	**±0.30 g**, < 40 dps
Soft-fall path	impact ≥ 1.80 g + posture change ≥ 55°
Auto-standby if not cleared	**60 s**

There is also a **soft-fall path** for a slump/faint that does not produce true weightlessness: a moderate impact plus a large posture change can confirm a fall. A bump with no posture change is rejected.

SOS FSM

State	Meaning
`IDLE`	monitoring
`ARMED`	a trigger is present but the confirmation timer is running
`ACTIVE`	emergency latched; OLED card + buzzer
`CANCELLED`	deliberate stand-down; latched until the condition clears

Triggers: **critical triage**, **fall**, **manual**, and (implemented but disabled by default) **prolonged contact loss**.

Parameter	Value
Critical triage confirmation	**3 s**

Parameter	Value
Fall grace/cancel window	**10 s**
Cancel hold	**5 s**
Recovery stand-down	configurable; critical condition must clear

Integration:

- `task_imu_monitor` reads the MPU6050 at **40 Hz**.
- `fall_detect_update()` runs the FSM.
- On confirmation, `sos_arm_fall()` starts the **10 s** grace period.
- The OLED shows the countdown and "HOLD CANCEL TO STOP"; the buzzer chirps during the countdown.
- If not cancelled, the SOS latches `ACTIVE`; the buzzer sounds and the rescue card is shown.
- A confirmed fall can be cleared by the SOS cancel path or the 60 s auto-standby.

Validation status: the fall FSM has **16/16 host unit tests passing** (free-fall timeout, impact without free-fall rejected, gyro gate, rebound settling, recovery motion, stillness reset, soft-fall, bed-rest baseline, auto-standby, null-pointer safety). The **physical drop/fall test on the real board is still pending**, so do not call the fall detector "field validated." It is implemented and unit-tested, not physically calibrated.

Important form-factor caveat: the fall FSM assumes the IMU is attached to the body. The current bench prototype is a fingertip/breadboard setup, not a worn device, so the fall path is implemented but not yet demonstrated as a wearable. The physical enclosure/strap is the missing link.

Safe judge answer:

> "Yes — a 5-state fall FSM: free-fall, impact, post-impact inactivity and confirmation, with a gyro gate and a soft-fall/posture path. It arms a 10-second SOS countdown with a 5-second cancel hold. The FSM has 16/16 host tests; the physical drop test is the remaining validation step."

---

13. Outputs and interfaces

OLED

- 128×64 SSD1306 over I²C.
- Shows HR, SpO_2, HRV state, risk, advisory, and connection/status.
- Works with no phone and no network.

USB serial

The PC sees structured lines:

- `[TELEMETRY]` — HR, SpO_2, RMSSD, temperature, humidity, PM2.5, RR, SQI, PI
- `[TRIAGE]` — NEWS2, level, flags, risk, rule/NN scores, PSI, AHA, cyclone/trend
- `[ENV]` — human-readable environmental summary
- `[LOC]` — stored location
- `[FPGA ACCEL]` — beat events

Web dashboard

- Standalone HTML/JS in `firmware/shrikefi/dashboard/valor_dashboard.html`.
- Connects over **Web Serial**; select **COM5** (port can change after replug).
- If it shows nothing after a reflash: refresh with `Ctrl+Shift+R`, click **Connect**, and choose the current COM port. Web Serial needs that user click; it cannot auto-select.

- No CDN and no internet dependency.
- Shows live PPG scope, HR/SpO_2/HRV/SQI/PI, environment strip, risk cards, CSV export, and simulated disaster profiles.
- With no finger, the correct state is **AMBIENT / NO CONTACT — ambient IR (not a pulse)**, not a fake waveform.

SoftAP page

- ESP32 hosts `VALOR-0081` at **192.168.4.1**.
- Phone/laptop can open the local status page without a router.
- Same-origin checks and a CSP are used; it is a prototype, not a hardened web server.

MQTT

- Optional developer path, **off by default** for privacy.
- Demo does **not** need it. Do not claim a production cloud backend.

24-hour history

- 24 hourly slots in **RAM**.
- Fills only as the device runs; resets on reboot.
- Labeled **“since boot”**, not a persistent calendar history.
- NVS persistence is not implemented; no time source / design decision yet.

---

14. What is verified vs what is not

Be precise. Judges reward this.

Claim	Status
Zynq PPG accelerator	**Verified**; 6/6 self-checking tests, timing closed
ForgeFPGA RTL	**Verified in simulation**; 13/13 link checks
Common peak-detector unit test	**4/4 passed**
ForgeFPGA fit	**515 LUT5 / 303 FF / 105 CLB**, 35 CLBs free
Bitstream flash	**46,408 bytes loaded** on COM5
Live HR / SpO_2 / SQI / PI	**Working** in live captures
HRV / RMSSD	**Not clinically validated**; split/startup artifacts still being fixed
SpO_2 accuracy	**Rough two-point calibration**, not clinical
Respiratory rate	**Often unresolved**; RSA from finger PPG is weak
Skin temperature	**Not fitted**; flood/hypothermia uses ambient proxy, capped at HIGH
Fall detection	**5-state FSM implemented; 16/16 host tests pass**; physical drop test still pending
SOS escalation	**4-state FSM implemented**; 10 s fall grace + 5 s cancel hold
24-hour history	**RAM only since boot**, not persistent
ForgeFPGA timing	**No real 50 MHz constraint**; 88.059 MHz is an unconstrained-fit indication
MQTT/cloud	**Dev only / off by default**
Medical device status	**Prototype; not a medical device**

The one HRV sentence to memorize

> “The FPGA timing is real and the beat detection works, but RMSSD is still under engineering validation — we have seen dicrotic-notch split pairs, and we need an independent ECG/chest strap before calling it clinically valid.”

Do not say “HRV is fixed” or “HRV is accurate” yet.

12.1 Six assumptions that are not true

1. **It is not a medical device.** It is a triage/engineering prototype.
2. **It is not a finished wearable.** It is a breadboard/USB bench prototype; no enclosure, battery or charger. The PPG is currently **fingertip contact**, not a wrist strap.
3. **It cannot call an ambulance.** There is no cellular modem or SMS path; SOS is local OLED + buzzer + local web. A bystander must be nearby to act on it.
4. **It does not keep a persistent 24-hour record.** The 24 slots are RAM-only since boot.
5. **It does not use a cloud AI.** TinyML runs on the ESP32; the cloud/MQTT path is optional and off by default.
6. **The fall FSM assumes the IMU is worn on the body.** The code and host tests exist, but the current bench prototype is not a worn device and the physical drop test is pending.

12.2 Live data vs simulated data — never mix them

- **Live hardware mode:** the dashboard connects to **COM5** over Web Serial; values come from `[TELEMETRY]` / `[TRIAGE]` lines produced by the board.
- **Simulation mode:** the dashboard has six synthetic disaster profiles. They are for UI demonstration and are labelled as simulated. Do not present a simulation profile as live sensor data.
- **MIMIC evidence:** the 94.11% figure is agreement with our triage rule on 16,387 ICU rows. HR and SpO_2 are real chart values; **RMSSD is synthesised** because MIMIC-III v1.4 has no waveform data; the labels are rule-derived. It is **not** independent clinical validation of the wearable.
- **24-hour chart:** the 24 hourly slots are accumulated since boot, not a calendar history; empty lanes show as gaps, not zero.

12.3 Physical build status — what is not fitted yet

Item	Status
Enclosure / strap	**Not fitted**; breadboard prototype, fingertip PPG, not a worn wrist device
Battery / charger	**Not fitted**; USB-powered
GPIO5 SOS cancel button	Firmware path ready; **physical button still pending**
MAX30102 reflective cradle	**Not printed**; SpO_2 uses a rough two-point calibration
TMP117 / skin-temperature sensor	**Not fitted**; flood/hypothermia uses ambient proxy, capped at HIGH
ECG / chest-strap reference	**Pending**; needed for HRV validation
Physical drop/fall test	**Pending**
NVS history	**Not implemented**; needs a time-source decision
Real 50 MHz FPGA constraint	**Still open**; current timing is unconstrained-fit indication
Cellular / emergency call	**Not present**

12.4 Failure modes and demo recovery

Symptom	What to do / what it means
Dashboard blank after reflash	Refresh `Ctrl+Shift+R`, **Connect → COM5**, close other serial monitors
No HR after finger	Wait 6–8 s; check finger contact; check `[PPG]` / `[FPGA ACCEL]`; power-cycle if FPGA is wedged
SpO_2 shows `CALC`	Wait 15–25 s; press firmer; check SQI/PI
No finger but red trace	Old build or wrong page; current build should show **AMBIENT / NO CONTACT**
GNSS `NO FIX`	Indoors/near window; stored deployment location is used as fallback
Wi-Fi/MQTT fails	Offline OLED, USB dashboard and SoftAP still work; MQTT is optional
Sensor NACK	Firmware retries I²C; UI shows `--`/UNKNOWN rather than a fake value
Low SQI	Device holds/qualifies the reading; critical vitals can still override
FPGA silent	Software peak-detector fallback after ~2.5 s; HRV may re-prime; do not hide this
Web page refuses a request	SoftAP page has same-origin checks; open it from the device AP, not an arbitrary host

12.5 Security and privacy — honest version

- The **SoftAP is open** (`WIFI_AUTH_OPEN`) so a rescuer can join without a password. It has **no authentication**; it is a local emergency-access page, not a hardened service.
- The page uses **same-origin checks and a CSP**, but that is not a full security audit.
- **MQTT is off by default**; the demo does not need the cloud.
- Data stays on the device by default; the stored location is local NVS text. Do not claim TLS or end-to-end encryption.
- Do not expose the SoftAP to an untrusted network or the public internet.

12.6 GNSS and location

- Receiver: **u-blox NEO-M8N / Quectel L89HS class**, UART2 at **9600 8N1**, NMEA-0183.
- Pins: **GPIO16 RX**, **GPIO17 TX**.
- A live 3D fix takes precedence; after **5 s** without a fix, the device falls back to the stored deployment location.
- There is **no map and no cellular uplink**; the OLED/page shows coordinates/text, not a rescue dispatch.
- Indoors, a GNSS fix is often unavailable. That is a physical limitation, not a firmware failure.

12.7 Reproduce the evidence in two minutes

```
# Firmware host suite (link, drivers, risk, web)
cd firmware/shrikefi
.\run_host_test.bat

# Dashboard and SoftAP regressions
python dashboard/verify_dashboard.py
python dashboard/verify_softap_page.py

# Fall FSM (16/16)
gcc -std=c11 -I. test_fall_detect.c fall_detect.c -lm -o "$env:TEMP\fall_test.exe"
& "$env:TEMP\fall_test.exe"

# ForgeFPGA simulation (13/13)
cd ../../hardware/shrikefi
build_shrikefi_sim.bat

# Bitstream/source consistency
python gen_flat_source.py --check
python convert_bitstream.py --check

# Rebuild and flash (ESP-IDF v5.5.x shell)
cd ../../firmware/shrikefi
idf.py build flash -p COM5
```

If a judge asks for a number you cannot reproduce on the spot, say **“I can show the file and the command, but I haven’t re-run it in this session.”** Do not invent a result.

13.8 If you do not know

Use this three-part answer:

1. **What is implemented** — point to the file.
2. **What is tested** — point to the test/result.
3. **What is not validated** — say it plainly.

Example:

> “The fall FSM is implemented in `fall_detect.c` and has 16 host tests. We have not done the physical drop test yet, so I would not call it field-validated.”

That answer is stronger than guessing.

---

15. Judge Q&A — short answers

Q1. What is VALOR?

> “An offline wearable triage companion that fuses PPG vitals with environmental hazards. The FPGA handles beat timing; the ESP32-S3 runs the clinical and disaster-risk logic; the OLED and local web page give an action without a network.”

Q2. Why an FPGA? Couldn’t the ESP32 do this?

> “It could, but not with deterministic timing. The FPGA gives a 50 MHz, 20 ns beat clock with no OS jitter and no Wi-Fi interrupt in the path. It also offloads the filter and peak detector, leaving the ESP32 for SpO_2, HRV, risk and UI.”

Q3. Why the ESP32-S3?

> “Dual-core, Wi-Fi/BLE, 8 MB flash, good FPU and low cost. Core 0 handles the 100 Hz optical loop; Core 1 handles environment, radio and UI.”

Q4. What sensors, and how are they connected?

> “MAX30102, BME280, MPU6050 and SSD1306 on I²C; PMS5003 and GNSS on UART; the ForgeFPGA on 4-wire SPI. All logic is 3.3 V; 5 V only powers the PMS5003.”

Q5. Why three different buses?

> “I²C is enough for short-board sensors and shares two wires. SPI gives the FPGA a deterministic one-byte-per-sample link. UART is the native output of the PMS and GNSS modules. Using the simplest bus for each job keeps the board and firmware clean.”

Q6. How is the FPGA programmed?

> “At boot, the ESP32 drives the FPGA power/enable/boot pins and streams the 46 KB bitstream over SPI2 at 16 MHz. The transfer is open-loop, so we don’t claim it as proof; the proof is the beat flag after real contact. It is not programmed over I²C.”

Q7. How do you get the beat and the IBI?

> “One SPI byte per sample: bit 7 is the beat flag, bits 6:0 are the filtered diagnostic. On a beat the FPGA queues five bytes: 0xC3 plus a 32-bit cycle count. The count is in 50 MHz ticks, so 20 ns resolution. Older bitstreams fall back to the ESP32 timer.”

Q8. How do you handle a noisy PPG?

> “Hardware: 8-tap moving average, 250 ms refractory, 25 % crest confirmation. Firmware: SQI from perfusion, skewness, interval regularity and IMU motion; if SQI is below 0.70 we hold/qualify instead of publishing a fake value. Critical vitals can still override.”

Q9. How is SpO_2 calculated?

> “Ratio of ratios: R = (AC_red/DC_red)/(AC_ir/DC_ir), then a calibrated curve. We smooth over 8 windows and wait for the estimate to stabilize. It is a prototype calibration, not clinical-grade.”

Q10. How is HRV calculated?

> “RMSSD over a 300-beat window of accepted beat intervals. It is real arithmetic on real intervals, but it is not clinically validated yet — split pairs and startup transients can still inflate it, and we have an ECG comparison pending.”

Q11. How do you get respiratory rate without a breathing sensor?

> “From RSA — respiratory sinus arrhythmia — in the beat intervals, using a Charlton-style estimator. Finger PPG RSA is weak, so the device reports confidence and often says ‘not resolved’ rather than printing a fake number.”

Q12. How does the risk score work?

> “mNEWS2 for the clinical side; heat, pollution, flood and cyclone rules from the environment; a 619-byte INT8 network that approximates the rule engine; then fusion takes the more severe risk. The rules are explainable; the NN is a compact approximation.”

Q13. Is the AI trained on patients?

> “No. The TinyML model is distilled from our rule engine, so it is compact and explainable. The MIMIC data is used for the triage/accuracy harness, not to claim the wearable is clinically validated.”

Q14. Does it work offline?

> “Yes. OLED advisory, USB dashboard and the local SoftAP page all work without internet. MQTT is optional and off by default.”

Q15. What about privacy?

> “The default build keeps data local. MQTT is a developer option and is disabled by default. The SoftAP is a prototype local page, not a hardened cloud service.”

Q16. What is your accuracy / validation?

> “We have self-checking RTL testbenches, a host test suite, live sensor captures, and a fit report. We do not claim clinical accuracy. HR and SpO_2 are live; HRV, RR and skin temperature still need independent validation or hardware.”

Q17. What is the power/battery life?

> “We have not measured it. The design is low-power by architecture — small FPGA, no DSP/BRAM, ESP32 radio dominant — but I won’t quote a mW or battery figure we haven’t measured.”

Q18. What happens if a sensor fails?

> “The firmware retries I^2C, clears stale values, and shows `--` or `UNKNOWN` rather than a fake normal. The risk engine holds or reports not assessed. That is deliberate: a reassuring fake value is worse than an honest gap.”

Q19. What is unique here?

> “The combination: health and environment fused on-device, FPGA-level beat timing, and a fully offline local dashboard. Most wearables are cloud-first and measure only the person.”

Q20. What would you do next?

> “Independent ECG/chest-strap HRV validation, proper SpO_2 calibration with the printed cradle, true-peak IBI timestamping / settling gate, NVS history with a real time source, a physical fall test, and a real 50 MHz FPGA constraint.”

Q21. Why both Zynq and ForgeFPGA?

> “Zynq is the verified baseline with AXI4-Lite and timing closure. ForgeFPGA is the low-cost wearable port using the same shared RTL over SPI. It shows the algorithm is portable, not tied to one vendor.”

Q22. Is this a medical device?

> “No. It is an engineering prototype and a triage aid. It is not a diagnostic device and has not gone through clinical validation.”

Q23. Can it call emergency services?

> “It has a local SOS state machine, buzzer and local web trigger. It does not have a cellular modem; it is not a replacement for emergency services. Location comes from GNSS and is shown locally.”

Q24. Why does HR take a few seconds?

> “The pipeline primes five beat intervals before it trusts the rhythm, then waits for the first accepted beat. With a normal resting heart rate that is about 6–8 seconds. It is not a hang.”

Q25. Why does SpO_2 take longer than HR?

> “SpO_2 is a ratio of two optical channels and it needs several 1-second windows to settle and stabilize. We wait for the estimate to stop moving instead of publishing the first number, so it takes roughly 15–25 seconds.”

Q26. Why does the dashboard show AMBIENT and no waveform with no finger?

> “That is deliberate. With no contact, the IR signal is ambient light, not a pulse. The firmware stops streaming raw PPG and the page shows AMBIENT / NO CONTACT rather than drawing a fake waveform.”

Q27. Why does HRV keep changing?

> “RMSSD is a 300-beat rolling window. It keeps incorporating new beats, so it moves; at 75 bpm it takes about four minutes to fill. Also, RMSSD is still under validation, so we do not present it as a stable clinical number.”

Q28. Does it detect falls?

> “Yes. A 5-state FSM: monitoring, free-fall, impact, post-impact inactivity, confirmed. It needs weightlessness, a high-g impact with rotation, and then sustained stillness — plus a soft-fall path using posture change for a slump or faint. It arms a 10-second SOS countdown.”

Q29. What happens after a fall?

> “The fall detector confirms, the SOS state machine moves to ARMED, the OLED shows a countdown, and the buzzer chirps. If nobody cancels with a 5-second hold within 10 seconds, it latches ACTIVE and shows the rescue card.”

Q30. Is the fall detector validated?

> “The FSM has 16/16 host unit tests. The physical drop test on the board is still pending, so I would call it implemented and unit-tested, not field-validated.”

Q31. How does it know the location?

> “A u-blox NEO-M8N on UART2 gives NMEA coordinates. A live fix takes precedence; if there is no fix for five seconds, the device falls back to the stored deployment location. It does not have a map or a cellular uplink.”

Q32. Is the local web page secure?

> “The SoftAP is intentionally open for emergency access and has no authentication; it is a local rescue page, not a hardened service. The page has same-origin checks and a CSP, and MQTT is off by default. We do not claim TLS or production-grade security.”

Q33. Is the dashboard showing real data?

> “In live mode, yes — it reads `[TELEMETRY]` / `[TRIAGE]` from COM5. It also has six deliberately synthetic disaster profiles for UI demonstration. We never present a synthetic profile as live hardware data.”

Q34. What is still missing for a production device?

> “A proper SpO_2 cradle and calibration, ECG/chest-strap HRV validation, a physical fall test, a real 50 MHz FPGA constraint, NVS history with a time source, an enclosure, battery/charger and power measurement, and a physical SOS button.”

Q35. Is the 24-hour history persistent?

> “No. It is 24 hourly slots in RAM, filled since boot. It resets on reboot and is labelled ‘since boot.’ NVS persistence needs a real time source, which we have not added yet.”

Q36. Is it wearable? How is it worn?

> “Not yet as a finished device. The current build is a breadboard bench prototype with a fingertip PPG. The wearable version needs a reflective cradle/strap and an enclosure; the fall FSM assumes the IMU is body-worn, so the physical fall test comes after that.”

Q37. What happens if the wearer is alone and unconscious?

> “The device can latch a local SOS and sound the buzzer, but it has no cellular modem or SMS path. It is a local alarm/triage aid, not an emergency dispatch system; a bystander or rescuer must be nearby.”

---

16. Demo script — 2 minutes

1. **Power on.** Show the boot log: - FPGA `configuration COMPLETE! (46408 bytes loaded)` - I²C devices: `0x3C`, `0x57`, `0x68`, `0x76` - sensors initialized.
2. **No finger.** Show OLED and dashboard: - `AMBIENT` - `NO CONTACT — ambient IR (not a pulse)` - temperature / humidity / PM2.5 still visible.
3. **Place finger.** - HR appears in ~6–8 s; - SpO_2 takes ~15–25 s to settle; - show SQI, PI, waveform.
4. **Open the dashboard** and select **COM5**: - live scope, vitals, risk cards; - CSV export; - simulation profiles for heat / pollution / flood.
5. **Optional SoftAP**: connect phone to `VALOR-0081`, open `192.168.4.1`.
6. **State the limits before the judge asks:** - HRV is under validation; - SpO_2 is prototype-calibrated; - 24 h history is since boot; - no battery measurement yet.

If the judge asks for HRV, show it but add the caveat immediately. Do not wait for the 300-beat window to settle during a short demo.

Pre-demo checklist

- ☐ Board on **COM5** (or the current port) and no other program holding it.
- ☐ Dashboard refreshed; **Connect → COM5** selected.
- ☐ No-finger state shows **AMBIENT / NO CONTACT**, with Ta/RH/PM2.5 visible.
- ☐ Finger session: wait for HR (~6–8 s) and SpO_2 (~15–25 s) before judging.
- ☐ Do not call HRV “validated” or “stable.”
- ☐ If the physical GPIO5 button is not fitted, demo SOS via the local web page and say so.
- ☐ GNSS may not fix indoors; know that the stored location is the fallback.
- ☐ Do not show a synthetic profile as live data.
- ☐ Keep the 3.3 V rule in mind; never probe with 5 V.

---

17. Numbers to memorize

Number	Meaning
100 Hz	PPG sample rate
50 MHz	FPGA clock
20 ns	FPGA timing tick
1 MHz	Runtime SPI clock

Number	Meaning
16 MHz	FPGA bitstream programming clock
46,408 bytes	Bitstream size
515 / 1120	LUT5s used
105 / 140	CLBs used; 35 free
88.059 MHz	Fitter-reported achievable frequency (unconstrained)
+8.644 ns	Margin at 50 MHz (unconstrained indication)
250 ms	Peak-detector refractory
25 %	Current crest fall confirmation
300	HRV beat window
619 bytes	INT8 TinyML model
0.70	SQI validity threshold
0.60 g / 2.50 g	Fall free-fall / impact thresholds
1.5 s	Post-impact stillness needed to confirm a fall
10 s / 5 s	Fall SOS grace / cancel hold
3.3 V	All logic; 5 V only PMS VCC

18. Glossary

Term	Meaning
PPG	Photoplethysmography — measuring blood-volume changes with light
IBI	Inter-beat interval — time between two heartbeats
HRV	Heart-rate variability — variation between beat intervals
RMSSD	Root mean square of successive differences — a short-term HRV metric
SpO_2	Estimated oxygen saturation of blood
PI	Perfusion index — pulsatile/steady optical ratio
SQI	Signal quality index — confidence in the PPG window
RSA	Respiratory sinus arrhythmia — breathing-related beat variation
mNEWS2	Modified NHS National Early Warning Score
PSI	Physiological Strain Index (Moran)
AHA strain	Autonomic-strain heuristic motivated by the AHA pollution statement
TinyML	Small neural network running on a microcontroller
INT8	8-bit quantized neural-network weights/activations
FSM	Finite-state machine — a fixed sequence of states and transitions
Free-fall	Near-weightless acceleration (< 0.60 g) before an impact
SOS	Local emergency escalation state machine (ARMED → ACTIVE)
SoftAP	ESP32-created Wi-Fi access point
Web Serial	Browser API used by the standalone dashboard to read COM5

Term	Meaning
NVS	ESP32 non-volatile storage (not used for history yet)
AXI4-Lite	Memory-mapped bus used by the Zynq baseline
LUT	FPGA lookup table; the logic fabric unit
CLB	Configurable logic block; the placement resource

19. Phrases to use / avoid

Use

- “engineering prototype”
- “triage aid”
- “not a medical device”
- “deterministic FPGA timing”
- “offline-first”
- “we hold rather than fabricate a value”
- “rule engine with a distilled TinyML approximation”
- “unconstrained fit indication, not timing closure”

Avoid

- “medical-grade”
- “clinically validated” / “100 % accurate”
- “the AI diagnoses”
- “24-hour persistent history”
- “5 V logic”
- “the FPGA does everything”
- “HRV is fixed”
- “SpO_2 is calibrated” without the prototype caveat

20. Where to read more

Topic	File
Master project guide	`docs/MASTER_PROJECT_GUIDE.md`
Detailed judge Q&A	`docs/JUDGE_QA.md`
SPI link protocol	`docs/SHRIKEFI_LINK_PROTOCOL.md`
Hardware debug handover	`docs/HARDWARE_DEBUG_HANDOVER.md`
Remaining work / limitations	`docs/REMAINING_WORK.md`
Firmware overview	`firmware/shrikefi/README.md`
ForgeFPGA synthesis evidence	`hardware/shrikefi/synthesis_evidence/README.md`
Dashboard behavior	`firmware/shrikefi/dashboard/README.md`

Final reminder: the strongest demo is not “everything works.” It is “here is what works, here is the evidence, and here is exactly what is not validated yet.” That is what distinguishes an engineering prototype from a pitch deck.